Integer Factorization as an Ising problem

Advith Desu

IIT Madras

May 5, 2026

Factorization Problem

- Given a natural number N, find P, Q such that

$$N = PQ$$

- Hard inverse problem underlying RSA security.
- Idea: map factorization $\rightarrow$ optimization problem.
- Solve using annealing-based techniques.

Binary Representation and Assumptions

Assumptions:

- N is odd.
- Factors have equal bit-length

$$n = \left\lceil \tfrac{1}{2} \log_2 N \right\rceil .$$

Binary form:

$$P = \sum_{k=0}^{n-1} 2^k p_k, \quad Q = \sum_{k=0}^{n-1} 2^k q_k, \quad p_k, q_k \in \{0, 1\}.$$

Fixed bits:

$$p_0 = q_0 = 1, \qquad p_{n-1} = q_{n-1} = 1.$$

Column-wise Multiplication

- Construct binary long multiplication table.
- Introduce carry variables $s_{i,j}$.
- Each column gives a constraint:

$$C_i = N_i - \sum_j q_j p_{i-j} - \sum s_{j,i} + \sum 2^j s_{i,i+j}.$$

- Correct factors satisfy $C_i = 0$.

Energy Function

Optimization objective:

$$\min_{\{p_i, q_i, s_{ij}\}} \sum_i (C_i)^2.$$

- Binary variables: factor bits + carry bits.
- Ground state corresponds to valid factorization.
- Alternative direct method:

$$\left(N - \sum p_i 2^i \sum q_j 2^j\right)^2$$

  but gives fewer preprocessing opportunities.

Rule 1: Saturated Sum

For binary $x_i \in \{0, 1\}$:

$$\sum_{i=1}^{n} x_i = n \quad \Longrightarrow \quad x_i = 1 \ \forall i.$$

Why: the only way n binary variables sum to n is for every one of them to be 1.

A symmetric form with negative coefficients:

$$\sum_{i=1}^{n} -x_i = -n \quad \Longrightarrow \quad x_i = 1 \ \forall i.$$

Rule 2: Zero Sum

For binary $x_i \in \{0, 1\}$:

$$\sum_{i=1}^{n} x_i = 0 \quad \Longrightarrow \quad x_i = 0 \ \forall i.$$

Why: a sum of non-negative binaries equals 0 only when every term is 0.

The same conclusion holds with negative coefficients:

$$\sum_{i=1}^{n} -x_i = 0 \quad \Longrightarrow \quad x_i = 0 \ \forall i.$$

Rule 3: Doubling Rule

For binary variables $x_1, x_2, x_3 \in \{0, 1\}$:

$$x_1 + x_2 = 2x_3 \quad \Longrightarrow \quad x_1 = x_2 = x_3.$$

Why: LHS $\in \{0, 1, 2\}$ and RHS $\in \{0, 2\}$. Equality forces LHS even, so $x_1 + x_2 \in \{0, 2\}$ — both equal, and equal to x_3.

Use in factorization: appears whenever a column clause has only two factor-bit terms balanced against a single carry, e.g.

$$p_k + q_k = 2s \;\Rightarrow\; p_k = q_k = s.$$

Rule 4: Coefficient Bound Rule

General linear constraint on $x_i \in \{0, 1\}$:

$$\sum_i c_i x_i + C = 0, \qquad S_+ = \sum_{c_i > 0} c_i, \quad S_- = \sum_{c_j < 0} |c_j|.$$

If a single coefficient is larger than the max possible opposing sum, the variable is *forced*:

Positive coefficients ($c_i > 0$):

$$c_i > S_- - C \;\Rightarrow\; x_i = 0, \qquad c_i > S_+ + C \;\Rightarrow\; x_i = 1.$$

Negative coefficients ($c_j < 0$, $|c_j|$ magnitude):

$$|c_j| > S_+ + C \;\Rightarrow\; x_j = 0, \qquad |c_j| > S_- - C \;\Rightarrow\; x_j = 1.$$

Example: $x_1 + x_2 = 2x_3 + 1$. Coefficient of x_3 is 2, max LHS is 2, so x_3 cannot be $1 \Rightarrow x_3 = 0$.

Rule 5: Complement Rule

For binary $x_1, x_2 \in \{0, 1\}$:

$$x_1 + x_2 = 1 \quad \Longrightarrow \quad x_2 = 1 - x_1, \quad x_1 x_2 = 0.$$

Why: the only solutions are $(0, 1)$ and $(1, 0)$ — exactly one variable is 1, so their product is always 0.

Use in factorization: eliminates one variable outright (substitute $x_2 \to 1 - x_1$) and kills any quadratic term $x_1 x_2$ that appears in a column clause.

Rule 6: Complementary Pair Rule

For a constraint $\sum_i c_i x_i + C = 0$ with S_+, S_- as before:

Two positive coefficients $c_i, c_j > 0$:

$$c_i + c_j > S_- - C \quad \text{and} \quad c_i + c_j > S_+ + C$$

$$\implies \quad x_j = 1 - x_i.$$

Two negative coefficients $c_i, c_j < 0$ (let $\bar{c} = |c|$):

$$\bar{c}_i + \bar{c}_j > S_+ + C \quad \text{and} \quad \bar{c}_i + \bar{c}_j > S_- - C$$

$$\implies \quad x_j = 1 - x_i.$$

Why: the first inequality forbids both variables from being 1 simultaneously; the second forbids both from being 0. The only remaining option is one of each.

Parity Rule

Both sides of a clause must have the same parity:

$$x_1 + x_2 + \text{even} = \text{even} \;\Rightarrow\; x_1 = x_2,$$

$$x_1 + x_2 + \text{even} = \text{odd} \;\Rightarrow\; x_1 + x_2 = 1.$$

Example:

$$x_1 + x_2 + 2x_3 = 2y_1 + 4y_2 \;\Rightarrow\; x_1 = x_2,$$

$$x_1 + x_2 + 2x_3 = 2y_1 + 4y_2 + 1 \;\Rightarrow\; x_1 + x_2 = 1.$$

Why it matters: column clauses balance a small sum of factor bits against carries with even coefficients ($2s_{i,i+1},\ 4s_{i,i+2}, \ldots$). The parity of the factor-bit sum is therefore locked by N_i.

Power Rule

For any binary variable $x \in \{0, 1\}$:

$$x^m = x, \quad m = 2, 3, \ldots$$

Why: $0^m = 0$ and $1^m = 1$ for all positive integers m.

Use in factorization: after squaring the energy function $E = \sum_i C_i^2$, every $p_k^2,\ q_k^2,\ s^2$ collapses back to its linear form $p_k,\ q_k,\ s$. This is what makes the squared objective expressible as a (degree-2) QUBO once cubics/quartics are quadrized.

Worked Example: $N = 391 = 17 \times 23$

Setup:

- $391 = 110000111_2$
- Fixed bits: $p_0 = q_0 = 1$ (odd), $p_4 = q_4 = 1$ (MSB)
- **Free bits:** p_1, p_2, p_3 and q_1, q_2, q_3 — 6 unknowns

Long multiplication table (cross-product $p_a q_b$ sits in column $a + b$):

	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
P					1	p_3	p_2	p_1	1
Q					1	q_3	q_2	q_1	1
					1	p_3	p_2	p_1	1
				q_1	$p_3 q_1$	$p_2 q_1$	$p_1 q_1$	q_1	
			q_2	$p_3 q_2$	$p_2 q_2$	$p_1 q_2$	q_2		
		q_3	$p_3 q_3$	$p_2 q_3$	$p_1 q_3$	q_3			
	1	p_3	p_2	p_1	1				
Carry-outs		$s_{6,7}, s_{6,8}$	$s_{5,6}, s_{5,7}$	$s_{4,5}, s_{4,6}$	$s_{3,4}, s_{3,5}$	$s_{2,3}, s_{2,4}$	$s_{1,2}$		
$N = 391$	1	1	0	0	0	0	1	1	1

Carry $s_{i,j}$ = bit of column i's carry destined for column j (coefficient 2^{j-i}).

Column Clauses: $C_i = 0$

Each column i: (column sum) + (carries in) $= N_i + 2s_{i,i+1} + 4s_{i,i+2}$.

$$
\begin{aligned}
C_0 &: \; 1 - 1 = 0 \quad \checkmark \\
C_1 &: \; p_1 + q_1 - 1 - 2s_{1,2} = 0 \\
C_2 &: \; p_2 + q_2 + p_1 q_1 + s_{1,2} - 1 - 2s_{2,3} - 4s_{2,4} = 0 \\
C_3 &: \; p_3 + q_3 + p_1 q_2 + p_2 q_1 + s_{2,3} - 2s_{3,4} - 4s_{3,5} = 0 \\
C_4 &: \; 2 + p_1 q_3 + p_3 q_1 + p_2 q_2 + s_{2,4} + s_{3,4} - 2s_{4,5} - 4s_{4,6} = 0 \\
C_5 &: \; p_1 + q_1 + p_2 q_3 + p_3 q_2 + s_{3,5} + s_{4,5} - 2s_{5,6} - 4s_{5,7} = 0 \\
C_6 &: \; p_2 + q_2 + p_3 q_3 + s_{4,6} + s_{5,6} - 2s_{6,7} - 4s_{6,8} = 0 \\
C_7 &: \; p_3 + q_3 + s_{5,7} + s_{6,7} - 1 - 2s_{7,8} = 0 \\
C_8 &: \; 1 + s_{6,8} + s_{7,8} - 1 = 0
\end{aligned}
$$

Applying Reduction Rules

C_8 **(Rule 2, sum $= 0$):** $s_{6,8} + s_{7,8} = 0 \Rightarrow \boxed{s_{6,8} = s_{7,8} = 0}$.

C_2 **(Rule 4, large coefficient):** LHS terms $p_2 + q_2 + p_1 q_1 + s_{1,2} \leq 4$, but coefficient of $s_{2,4}$ is 4. If $s_{2,4}$=1, need LHS ≥ 5. $\boxed{s_{2,4} = 0}$.

C_1 **(Rule 5):** $p_1 + q_1 = 1 + 2s_{1,2}$. Max sum $= 2 < 3$, so $s_{1,2} = 0$:

$$\boxed{s_{1,2} = 0, \quad q_1 = 1 - p_1} \quad \Rightarrow \quad p_1 q_1 = p_1(1 - p_1) = 0$$

C_2 **(after substitutions):** $p_2 + q_2 = 1 + 2s_{2,3}$. Same logic as C_1:

$$\boxed{s_{2,3} = 0, \quad q_2 = 1 - p_2}$$

C_7 **(Rule 2, sum $= 1$ exactly-one):**

$$p_3 + q_3 + s_{5,7} + s_{6,7} = 1 \quad (s_{7,8} = 0 \text{ already})$$

Exactly one of these four binaries equals 1.

Remaining Structure After Pre-processing

After the rules above, we have eliminated 5 variables ($s_{1,2}, s_{2,3}, s_{2,4}, s_{6,8}, s_{7,8}$) and reduced q_1, q_2 to functions of p_1, p_2.

Free QUBO variables (13):

$\{p_1, p_2, p_3, q_3\} \cup \{s_{3,4}, s_{3,5}, s_{4,5}, s_{4,6}, s_{5,6}, s_{5,7}, s_{6,7}\}$ (plus 2 auxiliaries from quadrization)

Remaining clauses (after substituting $q_1 = 1 - p_1,\ q_2 = 1 - p_2$):

$$
\begin{aligned}
C_3 :\ & p_3 + q_3 + p_1(1-p_2) + p_2(1-p_1) - 2s_{3,4} - 4s_{3,5} = 0 \\
& = p_3 + q_3 + p_1 + p_2 - 2p_1p_2 - 2s_{3,4} - 4s_{3,5} = 0 \\
C_4 :\ & 2 + p_1q_3 + p_3(1-p_1) + p_2(1-p_2) + s_{3,4} - 2s_{4,5} - 4s_{4,6} = 0 \\
& = 2 + p_1q_3 + p_3 - p_1p_3 + s_{3,4} - 2s_{4,5} - 4s_{4,6} = 0 \\
C_5 :\ & 1 + p_2q_3 + p_3(1-p_2) + s_{3,5} + s_{4,5} - 2s_{5,6} - 4s_{5,7} = 0 \\
C_6 :\ & 1 + p_3q_3 + s_{4,6} + s_{5,6} - 2s_{6,7} = 0 \\
C_7 :\ & p_3 + q_3 + s_{5,7} + s_{6,7} = 1
\end{aligned}
$$

Cubic terms appear after squaring (e.g. $p_1p_2 \cdot s_{3,4}$ in C_3^2, $p_1p_3 \cdot s_{4,5}$ in C_4^2) $\Rightarrow$ **quadrization required**.

Quadrization and QUBO Construction

Energy function: $E = \sum_{i=3}^{7} C_i^2$. Squaring introduces cubics like $a \cdot p_i p_j s_{k,\ell}$.

Rosenberg substitution: replace each AB pair appearing in a cubic by an auxiliary w, with penalty

$$P(A, B, w) = M\big(AB - 2Aw - 2Bw + 3w\big), \qquad M \gg \max |\text{coeff}|.$$

$P = 0$ iff $w = AB$. After substitution every cubic $AB \cdot s$ becomes $w \cdot s$ (quadratic).

Auxiliaries needed: $w_1 = p_1 p_2,\ w_2 = p_1 p_3$.

Full QUBO:

$$H = \sum_{i=3}^{7} C_i^2 \Big|_{w_1 \to p_1 p_2,\, w_2 \to p_1 p_3} + M\big[P(p_1, p_2, w_1) + P(p_1, p_3, w_2)\big]$$

Active QUBO variables (13 total):

Original (4)	$p_1,\ p_2,\ p_3,\ q_3$
Carry (7)	$s_{3,4},\ s_{3,5},\ s_{4,5},\ s_{4,6},\ s_{5,6},\ s_{5,7},\ s_{6,7}$
Auxiliary (2)	$w_1 = p_1 p_2,\ w_2 = p_1 p_3$

QUBO Matrix Structure

QUBO form: $H = \sum_i h_i x_i + \sum_{i<j} Q_{ij} x_i x_j + \text{const.}$

Example: contribution of C_1^2 before C_1 was eliminated (illustrative pattern):

$$C_1^2 = (p_1 + q_1 - 1 - 2s_{1,2})^2 \xrightarrow{x^2=x} -p_1 - q_1 + 8s_{1,2} + 2p_1q_1 - 4p_1s_{1,2} - 4q_1s_{1,2} + 1.$$

Sketch of nonzero entries of Q for $N = 391$ (qualitative):

	p_1	p_2	p_3	q_3	$s_{3,4}$	$s_{3,5}$	$s_{4,5}$	$s_{4,6}$	$s_{5,6}$	$s_{5,7}$	$s_{6,7}$	w_1	w_2
p_1	•	•	•	•								•	•
p_2		•	•	•	•	•						•	
p_3			•	•	•		•	•			•		•
q_3				•			•	•	•	•	•		
$s_{3,4}$					•	•	•	•					
$s_{3,5}$						•	•		•	•			
$s_{4,5}$							•	•	•	•			
$s_{4,6}$								•	•		•		
$s_{5,6}$									•	•	•		
$s_{5,7}$										•	•		
$s_{6,7}$											•		
w_1												•	
w_2													•

Ground state at $(p_1, p_2, p_3, q_3) = (0, 0, 0, 0)$ recovers P=17, Q=23 with E=0.

J matrix Visualization I

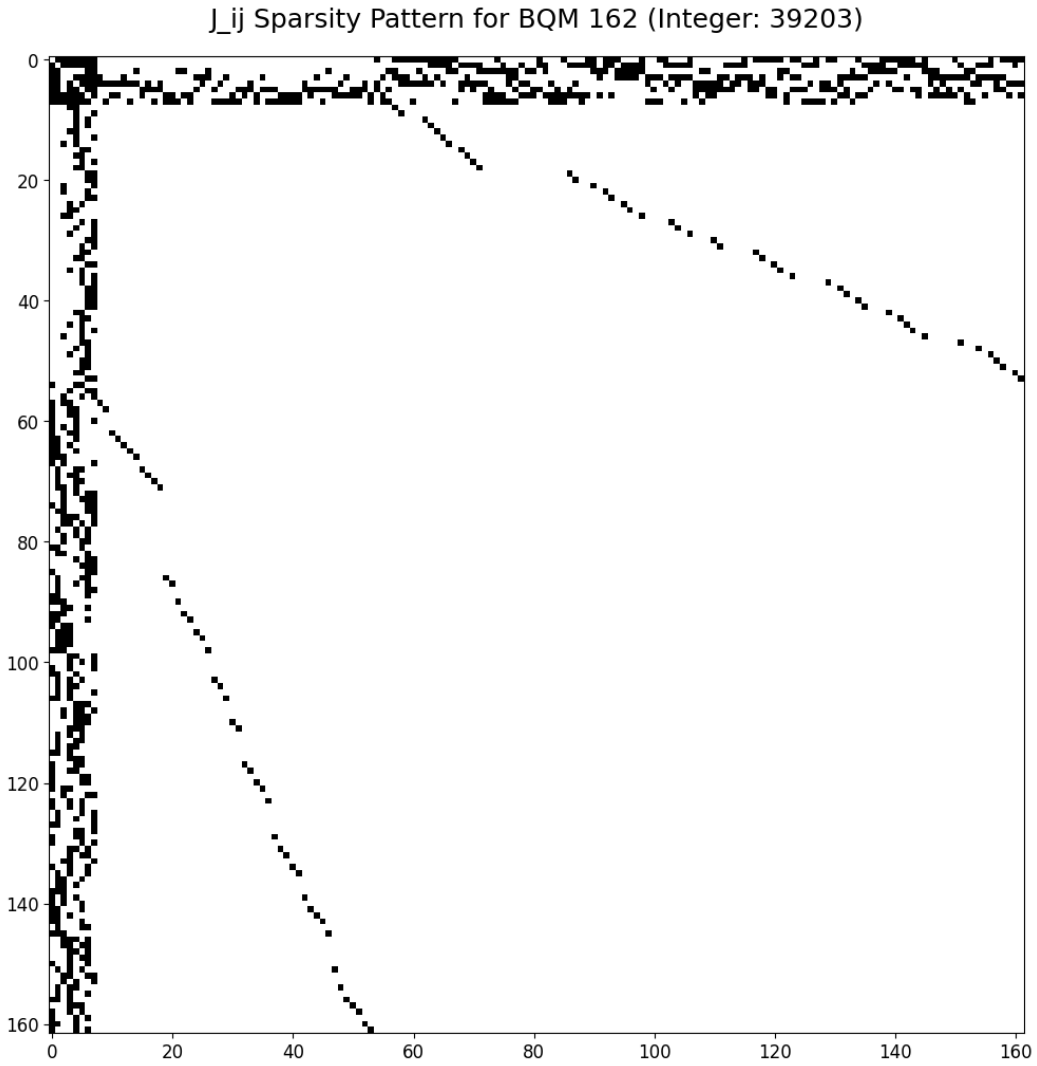


Figure: Sparsity of J matrix for 162 BQM (Integer 39203).

Sparse Coupling Matrix Representation (CSR)

Problem: The Ising coupling matrix J_{ij} is very large but mostly zero. Storing it as a dense $N \times N$ matrix wastes memory and bandwidth.

Key idea: Store *only the nonzero couplings* using Compressed Sparse Row (CSR) format. CSR represents a sparse matrix using three arrays:

$$\texttt{row_ptr}[i] \quad \text{start index of row } i$$

$$\texttt{col_idx}[k] \quad \text{column index of nonzero entry}$$

$$\texttt{values}[k] \quad \text{value of } J_{ij}$$

Only nonzero entries are stored.

Memory cost: $\mathcal{O}(N + \text{nnz})$ instead of $\mathcal{O}(N^2)$

Graph-Coloring Simulated Annealing

In one Metropolis sweep we want to flip many spins in parallel, but two spins coupled by a non-zero J_{ij} share an edge and cannot be flipped simultaneously without invalidating each other's ΔE.

We use graph coloring on the interaction graph (nodes = spins, edges = nonzero J_{ij}) partitions spins into colors such that *no two same-color spins are coupled*. All spins in one color can then be flipped in parallel without affecting each other's energy calculations.

Two performance bins based on row degree $d_i = |\{j : J_{ij} \neq 0\}|$:

- **Dense** ($d_i \geq 128$): full block of 1024 threads, shared-memory reduction.
- **Sparse** ($d_i < 128$): one warp (32 threads) per spin, 32 spins packed per block, warp-shuffle reduction.

One-time Setup (Host)

Algorithm Pre-processing on host before annealing

Require: CSR of symmetric J, linear field h, num_spins N

1: Verify J symmetry and a zero diagonal
2: Bin each spin: **dense** if $d_i \geq 128$, else **sparse**
3: $\textsc{color}[\,] \leftarrow$ Welsh–Powell greedy coloring (largest-degree-first, first-fit)
4: Build sparse-only CSR with hub-tagged `col_idx` (high bit $\Rightarrow$ index into hub cache)
5: Bucket spins by color: `color_dense_offsets`, `color_sparse_offsets`
6: Upload all CSR + color tables + hub arrays to GPU
7: Initialize a random spin configuration and compute initial energy
8: Calculate the start-temperature: $T_0 = \langle \Delta E^+ \rangle / \ln(1/\chi)$

Annealing Loop (GPU)

Algorithm Color-parallel simulated annealing

Require: β-schedule $\{\beta_t\}$ (geometric), sweeps per β, N_{colors}

```
for β in β-schedule do
    for sweep = 1 ... m do
        Generate fresh uniform randoms for all N spins
        for c = 0 ... N_colors − 1 do                    ▷ colors run sequentially
            collectFlipCandidates⟨dense⟩ on color c
            collectFlipCandidates⟨sparse⟩ on color c
                each spin i in color c in parallel:
                    L_i ← Σ_j J_ij s_j + h_i, ΔE_i ← −2 s_i L_i
                    if rand_i < min(1, e^(−β ΔE_i)): append (i, ΔE_i) to candidates
            applyAllFlipsInColor: flip every accepted spin, atomicAdd ΔE_i
to total energy
            if any hub spin in color c: refresh hub-values cache
        end for
    end for
    if total energy < best energy then snapshot best spins
    end if
end for
return best-energy spin configuration
```

GPU Kernel

Digital Annealer 1 (DA1) manually launches kernels by assigning specific threads to do the energy calculations. See GitHub Link for the code used.

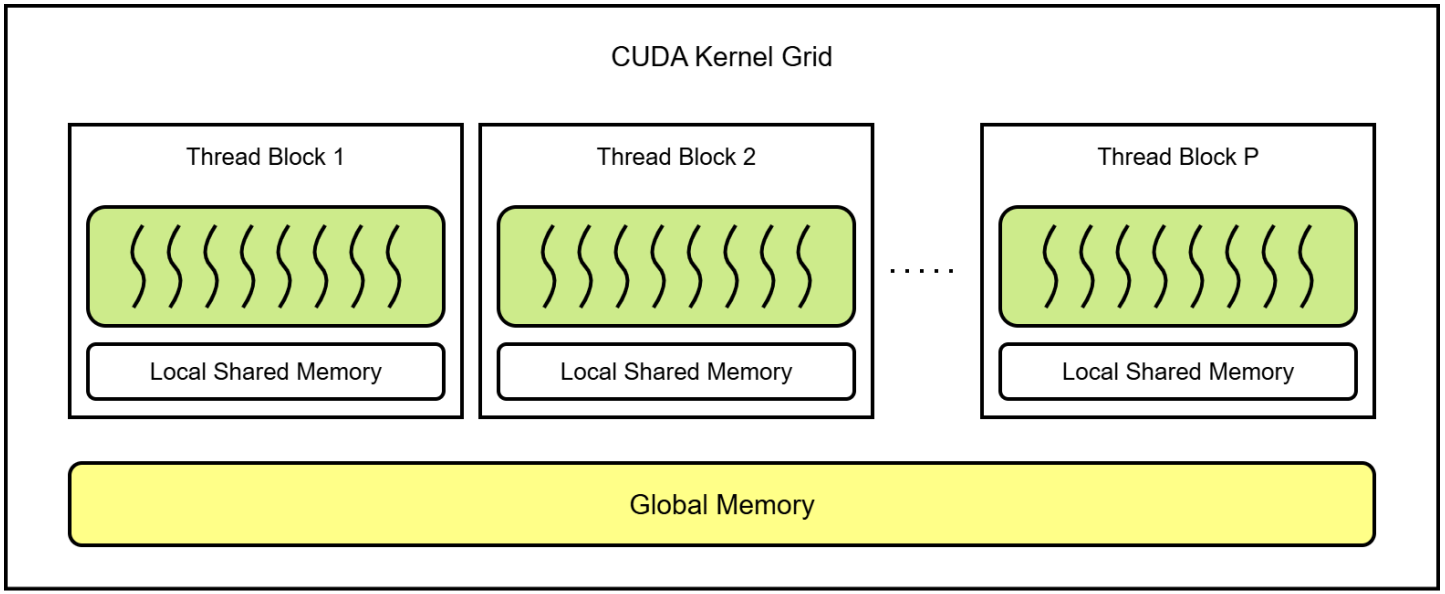


Figure: GPU Threads and Thread blocks.

GPU Thread Calculation in DA1

Each thread i calculates the local field $L_i = \sum_j J_{ij} s_j + h_i$ and the energy difference is then calculated as $-2 \times s_i \times L_i$ depending on which i^{th} spin flipped.

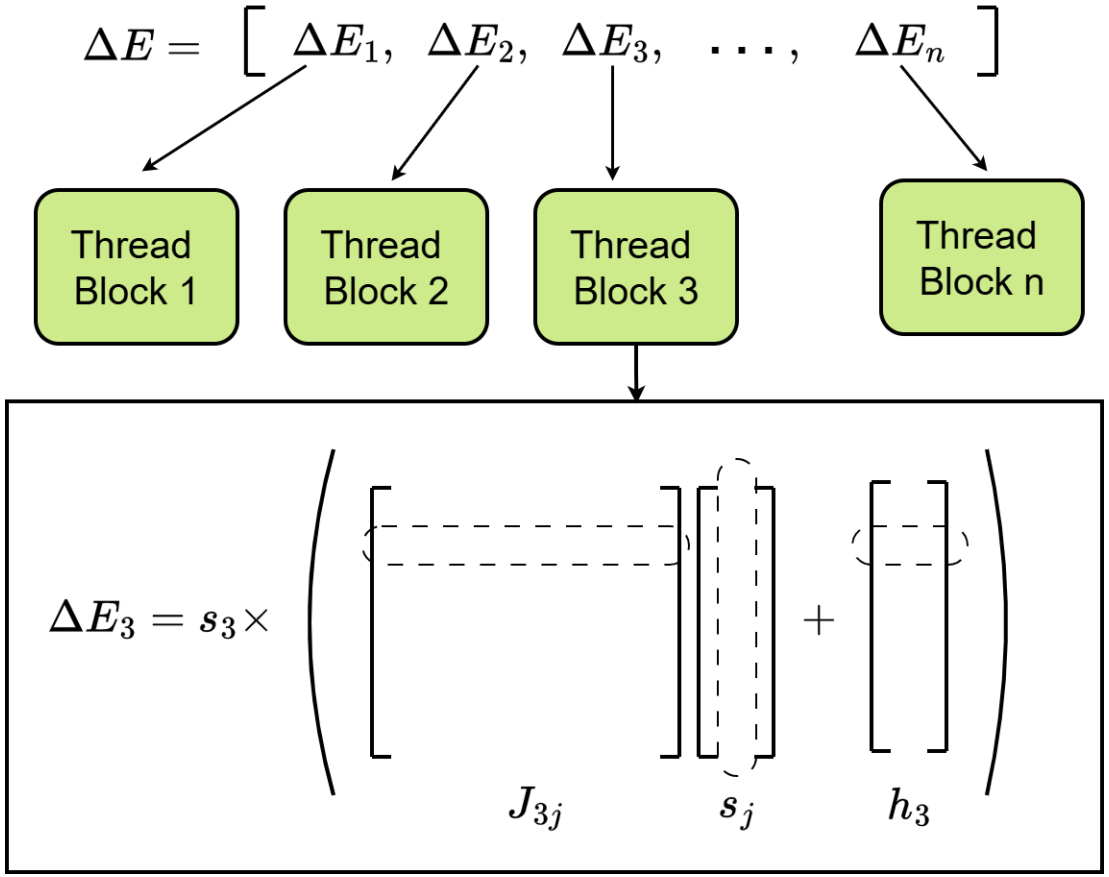


Figure: Energy calculation in DA1.

Number of Densely connected spins with scale

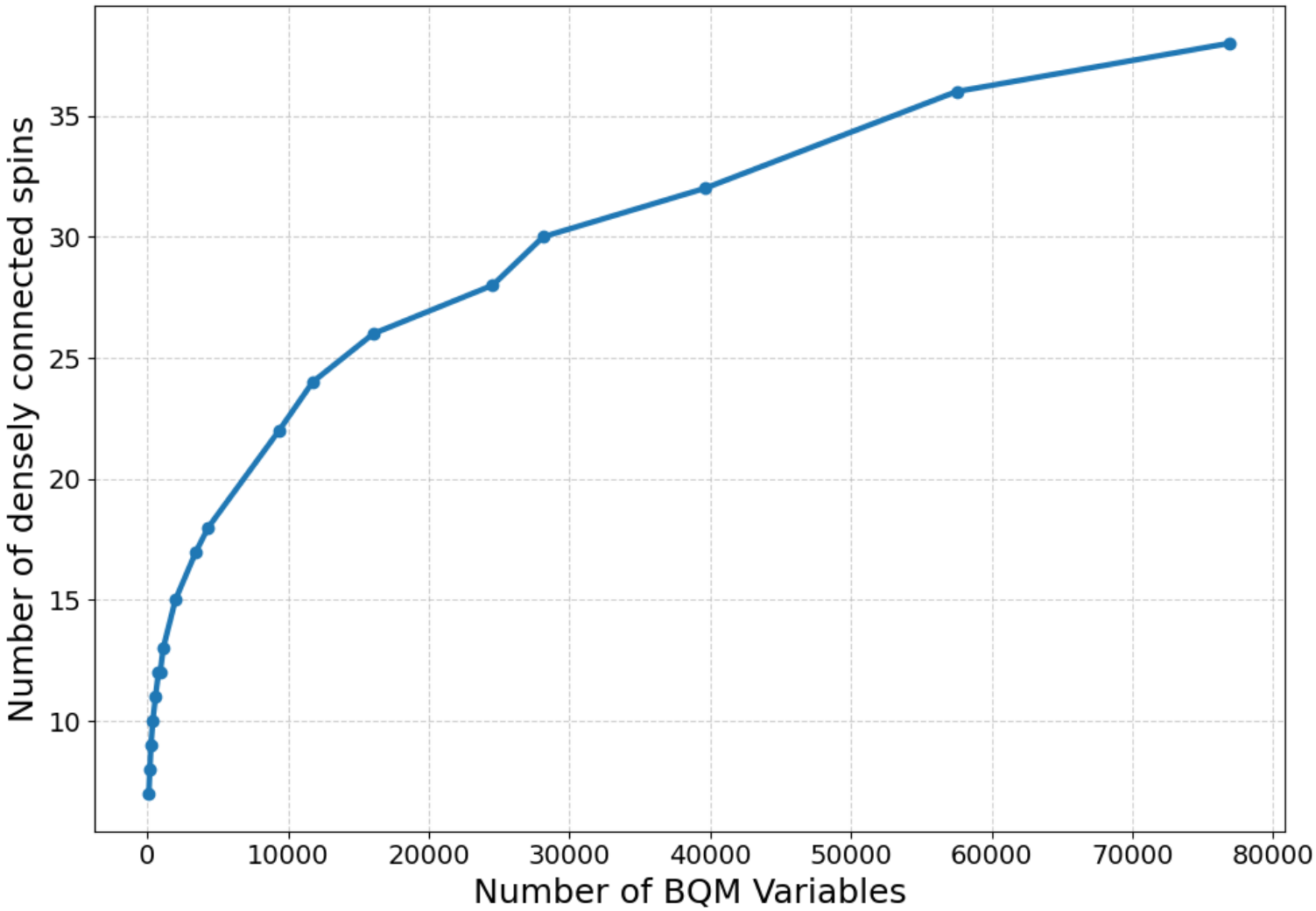
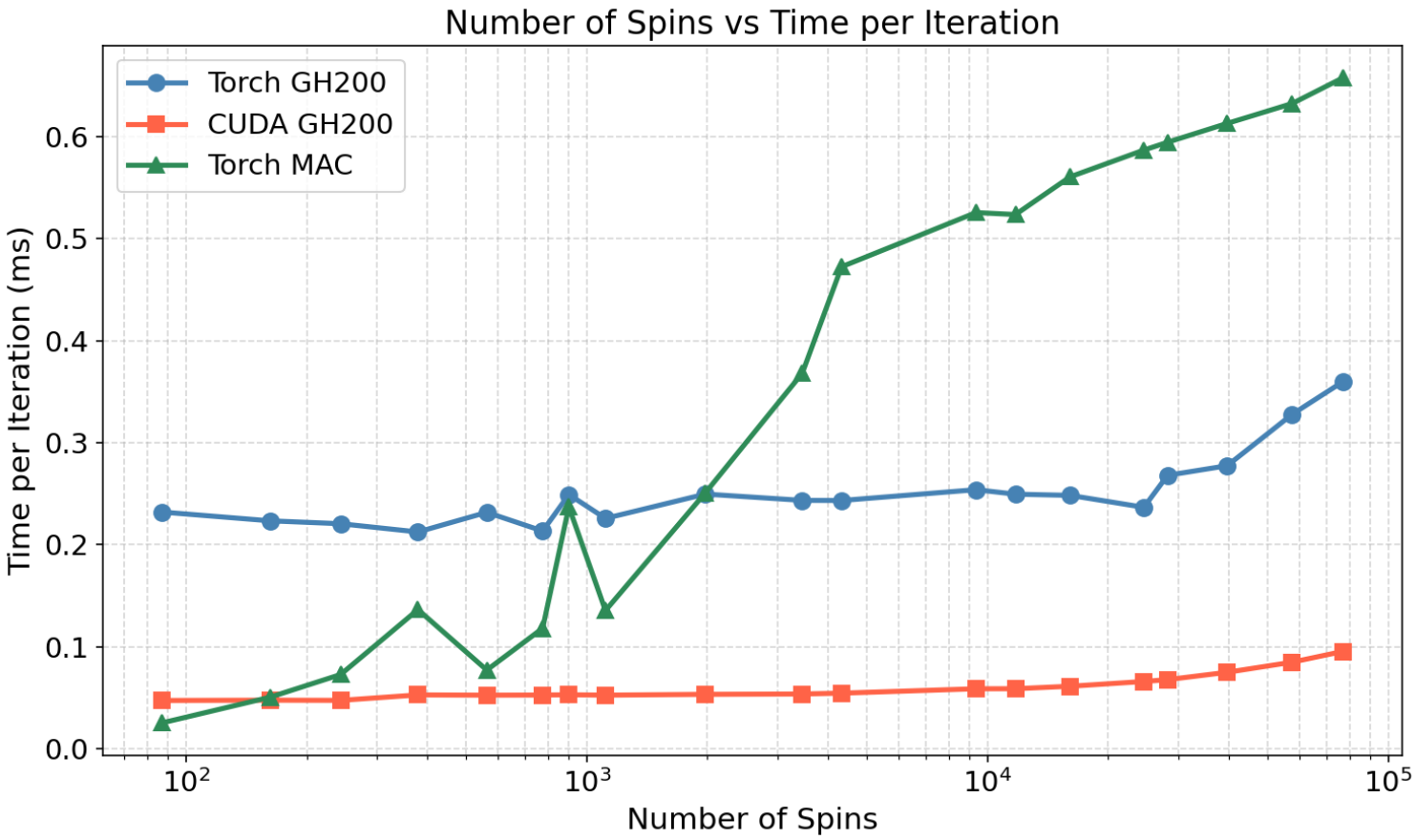
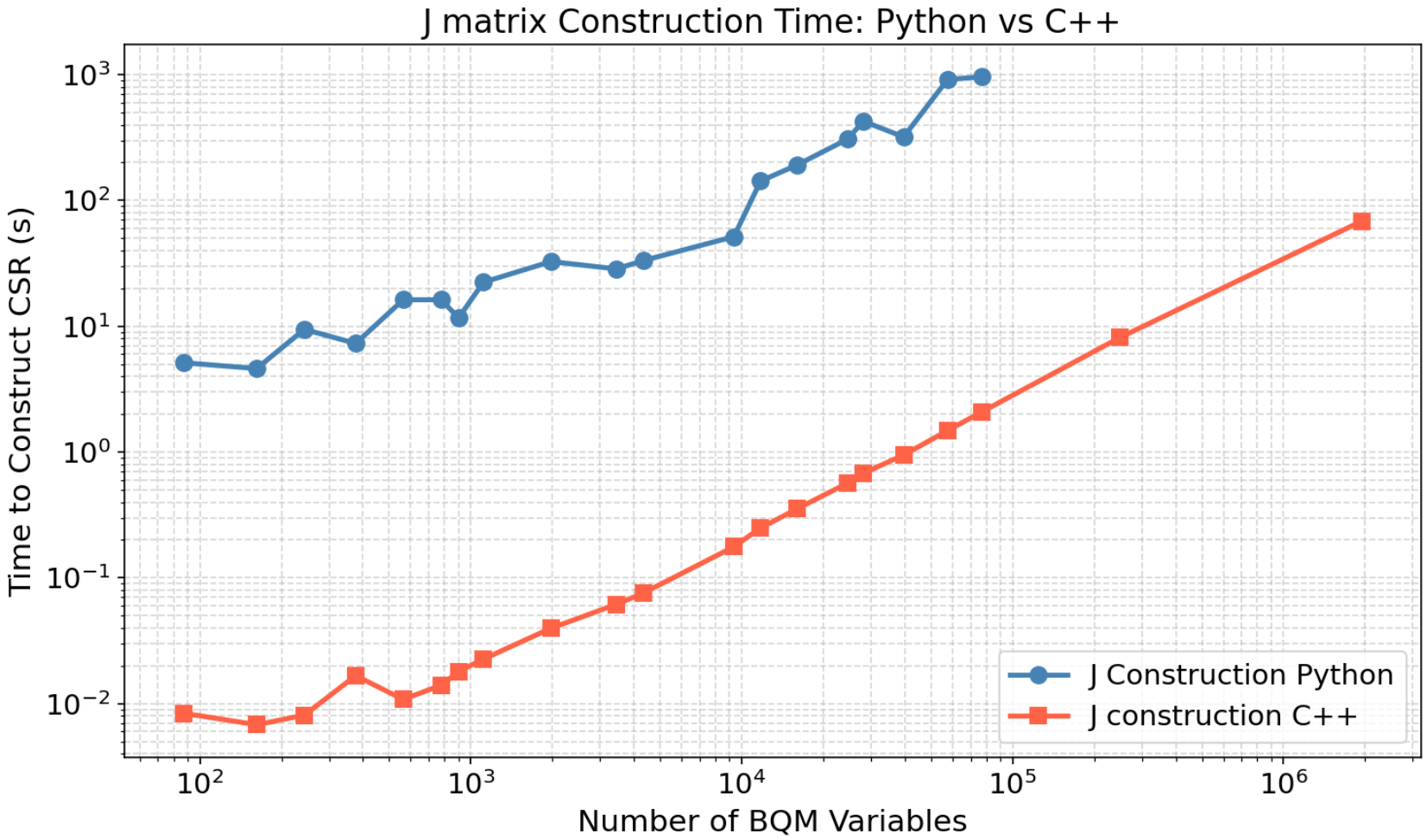


Figure: BQM variables vs number of densely connected spins.

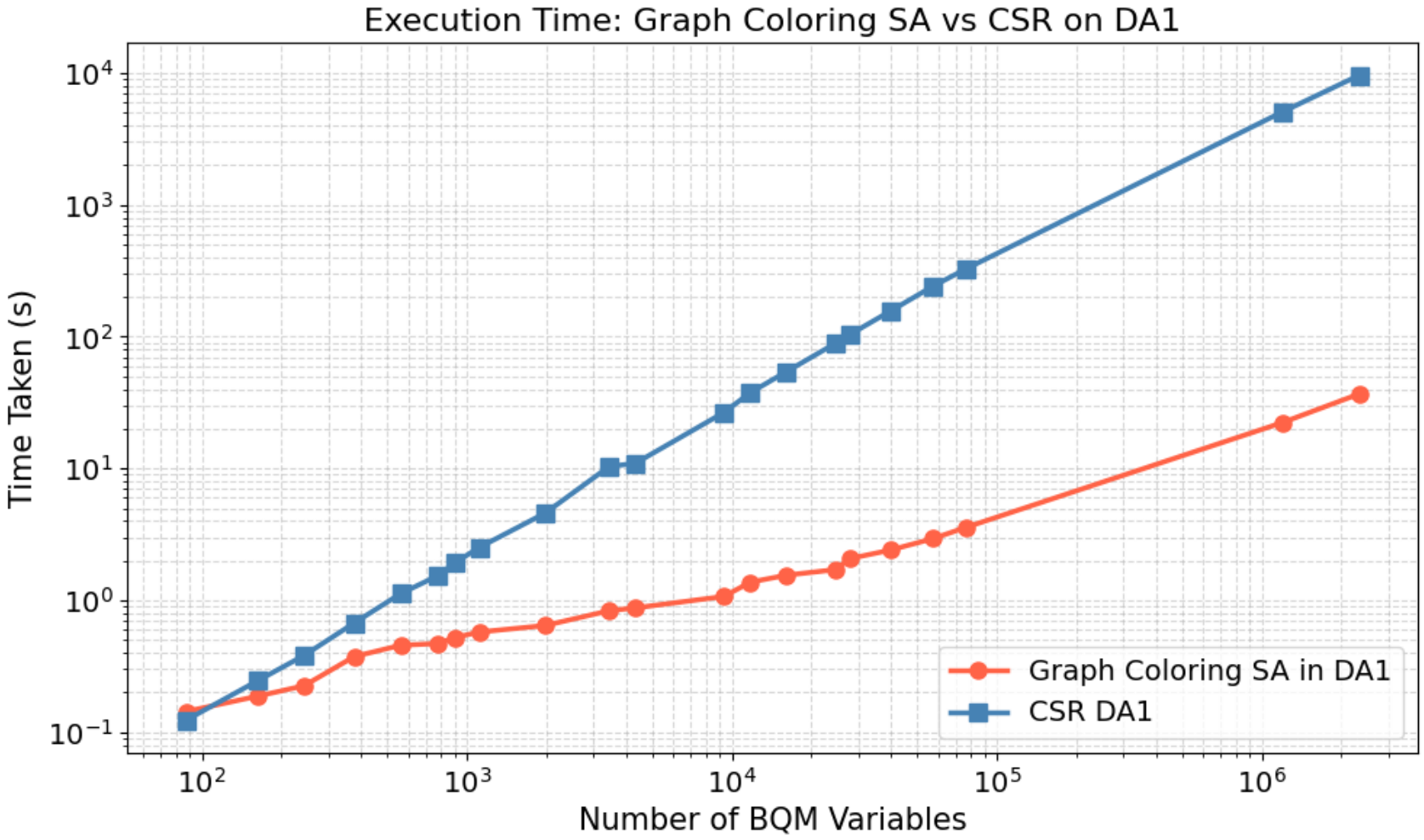
Annealing Time per Iteration vs Number of Spins



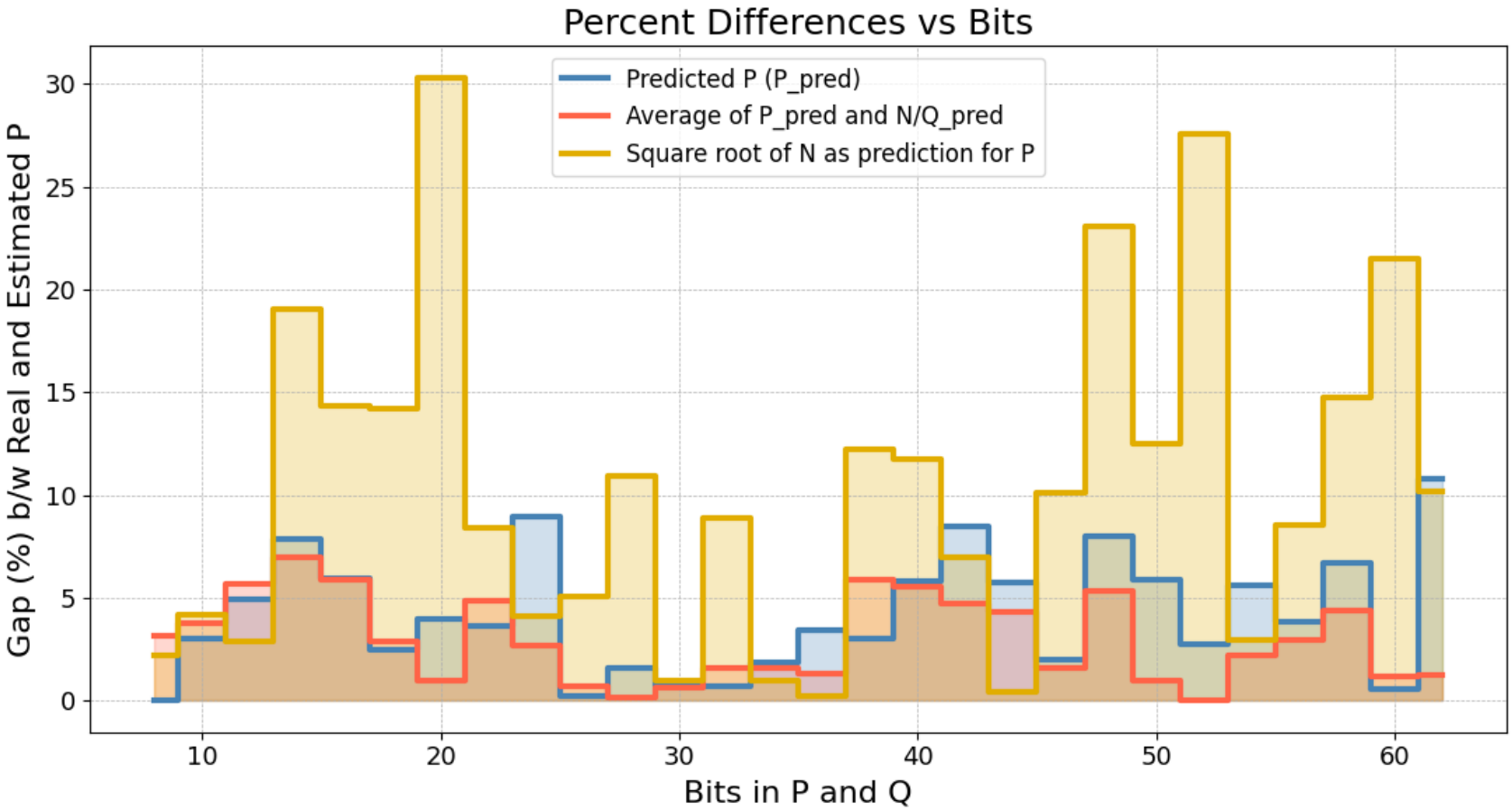
Construction time for CSR J



Execution Time Comparison

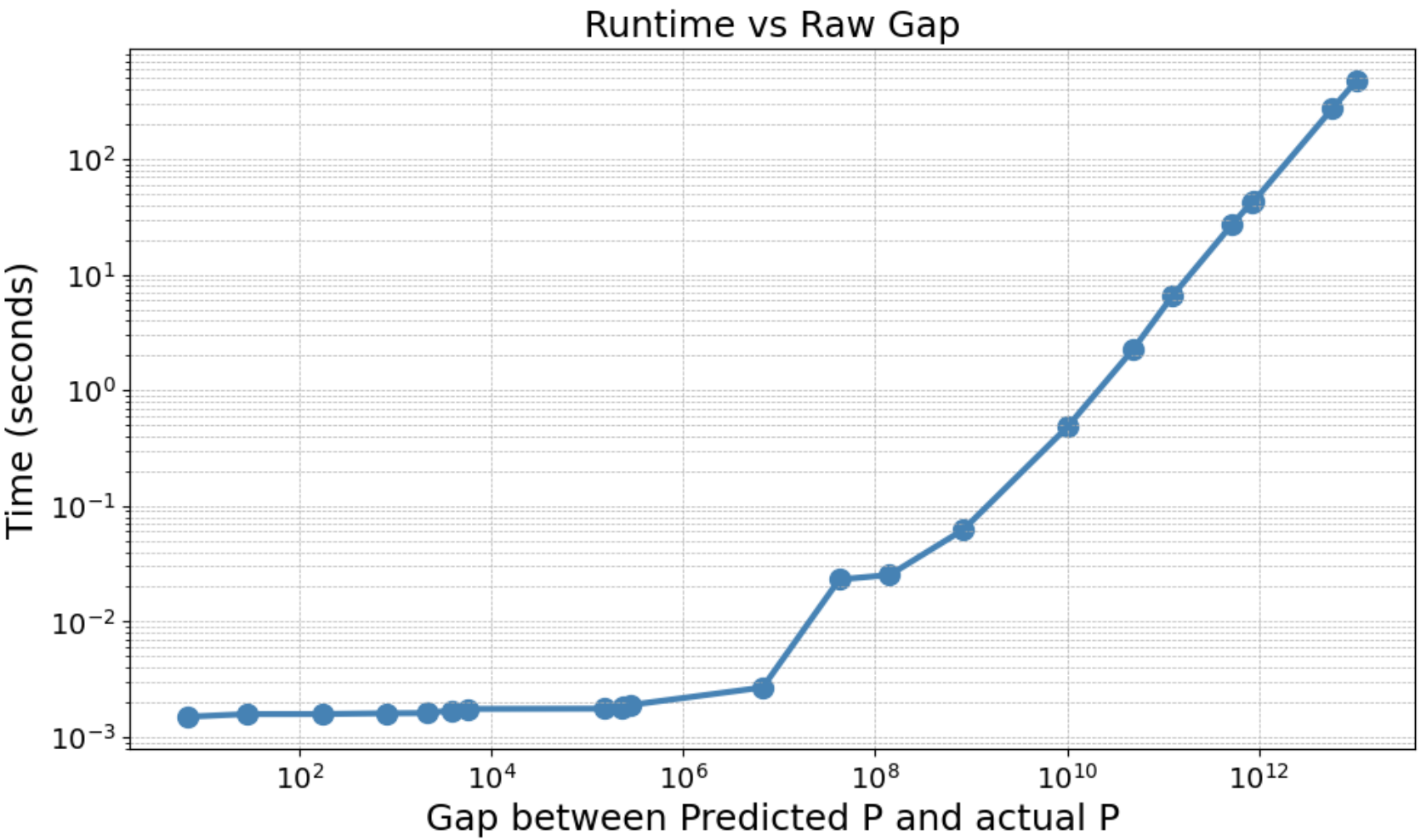


Solution Quality Measured by Gap between Real and Estimated P



1. Average (%) difference with square root N as the estimate = **10.33%**
2. Average (%) difference with P_{pred} from Simulated Annealing = **4.23%**
3. Average (%) difference with mean of P_{pred} and N/Q_{pred} = **2.98%**

Time Taken to Find Exact primes by Brute Force



$$\text{Time Taken (s)} = \frac{3.32 \times 10^{-9}}{\text{Threads}} \times \text{Gap (\%)} \times 2^{n}$$